校园失物招领系统 - 后端开发指南

一、项目概览

技术栈

技术	用途
Java 17+	编程语言
Jakarta Servlet 6.0	Web 框架（原生 Servlet，无 Spring）
Tomcat 10+	Servlet 容器 / 应用服务器
MySQL 8.0	关系型数据库
JDBC	数据库连接（`java.sql`）
JWT (io.jsonwebtoken)	用户身份认证（Token 生成与解析）
自研 JSON 工具	`JsonUtil.java` — 无第三方 JSON 依赖

目录结构

```
backend/
├── sql/
│   ├── init.sql            # 数据库建表脚本
│   └── test_data.sql       # 测试数据
├── web/
│   └── uploads/            # 图片上传存储目录
├── src/
│   └── com/lpo/
│       ├── filter/
│       │   └── CorsFilter.java    # CORS 跨域过滤器
│       ├── servlet/
│       │   ├── AuthServlet.java   # 认证相关（登录/注册/个人信息/排行榜）
│       │   ├── ItemServlet.java   # 物品相关（CRUD/搜索/匹配/看板统计）
│       │   ├── MessageServlet.java # 消息相关（收发/已读/会话）
│       │   ├── CategoryServlet.java # 分类列表
│       │   ├── AdminServlet.java  # 管理员端（登录/认证审核/用户管理）
│       │   └── UploadServlet.java # 文件上传
│       └── util/
│           ├── DbUtil.java        # 数据库连接池工具
│           ├── JsonUtil.java      # 自研 JSON 解析/序列化
│           ├── JwtUtil.java       # JWT 生成/解析/验证
│           └── ResponseUtil.java  # 统一响应格式封装
```

二、核心架构

2.1 请求路由

所有 Servlet 通过 `@WebServlet` 注解注册，URL 格式统一：

Servlet	URL 模式	职责
`AuthServlet`	`/api/auth/*`	用户登录、注册、个人信息、信誉排行榜
`ItemServlet`	`/api/items/*`	物品 CRUD、搜索、智能匹配、看板统计
`MessageServlet`	`/api/messages/*`	消息收发、已读标记、未读计数
`CategoryServlet`	`/api/categories`	分类列表查询
`AdminServlet`	`/api/admin/*`	管理员登录、认证审核、用户管理
`UploadServlet`	`/api/upload`	文件上传（Multipart）

每个 Servlet 的 `doGet`/`doPost`/`doPut`/`doDelete` 方法内部通过 `pathInfo` 正则匹配分发到具体处理方法。

2.2 统一响应格式

所有 API 返回标准 JSON 结构：

```
{
    "code": 200,          // HTTP 状态码
    "message": "success", // 提示信息
    "data": { ... }       // 业务数据（可选）
}
```

`ResponseUtil.java` 提供 5 个静态方法：

- `success(resp, data)` — 200 成功 + 数据
- `success(resp)` — 200 成功（无数据体）
- `error(resp, code, message)` — 自定义错误
- `unauthorized(resp)` — 401 未授权
- `forbidden(resp)` — 403 无权限

2.3 数据库连接

`DbUtil.java` 管理 JDBC 连接，配置从 `web/WEB-INF/web.xml` 的 context-param 读取（或使用默认值）：

```
// 默认配置
String url = "jdbc:mysql://localhost:3306/lost_property?
useUnicode=true&characterEncoding=utf8";
String user = "root";
String password = "root";
```

所有数据访问层使用 try-catch-finally + `DbUtil.close(conn, ps, rs)` 确保连接释放。

2.4 认证与授权

JWT 流程：

1. 登录成功 → `JwtUtil.generateToken(userId, username, isAdmin)` → 生成 Token
2. 客户端每次请求带 `Authorization: Bearer <token>`
3. 业务 Servlet 调用 `JwtUtil.getUserId(token)` 解析用户 ID
4. 管理端接口额外调用 `JwtUtil.isAdmin(token)` 验证管理员身份

Token 结构：

- `sub`：用户 ID
- `name`：用户名
- `admin`：是否管理员（`true`/`false`）
- `exp`：过期时间（24 小时）

2.5 CORS 跨域

`CorsFilter.java` 为所有请求添加跨域响应头：

```
response.setHeader("Access-Control-Allow-Origin", "*");
response.setHeader("Access-Control-Allow-Methods", "GET, POST, PUT, DELETE,
OPTIONS");
response.setHeader("Access-Control-Allow-Headers", "Content-Type, Authorization");
```

三、功能模块详解

3.1 用户认证（AuthServlet）

路由分发：

```
GET  /api/auth/profile           → 获取当前用户信息（需 Token）
POST /api/auth/login             → 用户登录
POST /api/auth/register          → 用户注册
PUT  /api/auth/profile           → 更新用户信息
POST /api/auth/certification     → 提交校园认证
GET  /api/auth/certification/list → 获取认证列表（管理员）
POST /api/auth/certification/review → 审核认证（管理员）
POST /api/auth/reputation        → 更新信誉积分（管理员）
GET  /api/auth/reputation/rank   → 获取热心值排行榜（公开）
GET  /api/auth/users             → 获取用户列表（公开信息）
```

注册逻辑：

1. 检查用户名是否已存在
2. 插入 `users` 表（默认 `reputation=100`, `is_certified=0`, `is_admin=0`）
3. 生成 JWT，返回 token + 用户信息

3.2 物品管理（ItemServlet）

物品类型存储：数据库存 `'found'` / `'lost'`，返回前端转换为 `1`（招领）/ `2`（寻物）。

`itemToMap` **方法**：统一的 ResultSet → Map 转换，处理类型、分类、状态等字段的格式转换。

分页查询：

```
GET /api/items?page=1&pageSize=12&type=1&categoryId=3&keyword=手机
&startTime=...&endTime=...
```

先查 `COUNT(*)` 得总数，再查分页数据，返回 `{ list, total, page, pageSize }`。

智能匹配（`GET /api/items/match/{id}`）：

1. 查当前物品的 type 和 category
2. 找相反类型（招领↔寻物）、同分类的物品
3. 优先同分类，其次按创建时间倒序，limit 5

全局看板统计（`GET /api/items/dashboard/stats`，公开）：

- 一次请求执行 4 条 SQL 聚合查询：
  1. 总发布量 + 已解决数 + 招领/寻物分布
  2. 今日新增数（`DATE(created_at) = CURDATE()`）
  3. 近 7 天每日新增趋势（`GROUP BY DATE(created_at), type`）
  4. 分类统计 Top 10（`LEFT JOIN categories` 获取分类名称）
- 返回统一 data 对象给前端渲染 ECharts

3.3 消息系统（MessageServlet）

核心 SQL 设计：

```
-- 所有查询方法统一 JOIN u_sender + u_receiver + LEFT JOIN items
SELECT m.*,
       u_sender.username as sender_username, ...
       u_receiver.username as receiver_username, ...
       i.title as item_title
FROM messages m
JOIN users u_sender ON m.sender_id = u_sender.id
JOIN users u_receiver ON m.receiver_id = u_receiver.id
LEFT JOIN items i ON m.item_id = i.id
WHERE ...
```

`buildMessageMap` **方法**：将数据库下划线字段映射为驼峰字段（`sender_id` → `senderId`），前端直接使用。

发送消息（POST）：

1. 兼容 `receiverId` / `receiver_id` 和 `itemId` / `item_id` 两种命名
2. 使用 `Statement.RETURN_GENERATED_KEYS` 获取自增 ID

3. 回查完整消息对象返回前端

标记已读：

- `PUT /api/messages/{id}/read` — 标记单条
- `PUT /api/messages/read` — 批量全标已读

3.4 文件上传（UploadServlet）

```
POST /api/upload (multipart/form-data, field: "file")
```

1. 获取 `Part filePart = req.getPart("file")`
2. 生成 UUID 文件名保留扩展名
3. 保存到 `web/uploads/`（或临时目录降级）
4. 返回 `{ url: "/uploads/uuid.png", fileName: "uuid.png" }`

四、数据库表结构

users 表

字段	类型	说明
id	int	主键
username	varchar(50)	用户名
password	varchar(100)	密码（明文）
nickname	varchar(50)	昵称
reputation	int	信誉积分（默认 100）
is_certified	tinyint(1)	是否已认证
is_admin	tinyint(1)	是否管理员

items 表

字段	类型	说明
id	int	主键
user_id	int	发布者 ID
type	varchar(10)	`'found'` 或 `'lost'`
status	varchar(20)	`'active'`/`'completed'`/`'deleted'`
category	varchar(20)	分类 ID，外键到 categories
title	varchar(100)	物品名称

字段	类型	说明
description	text	物品描述
location	varchar(200)	地点
contact	varchar(100)	联系方式
images	text	JSON 数组字符串，如 `["/uploads/a.png","/uploads/b.png"]`

messages 表

字段	类型	说明
id	int	主键
sender_id	int	发送者 ID
receiver_id	int	接收者 ID
item_id	int	关联物品 ID（可空）
content	text	消息内容
is_read	int	0 未读 / 1 已读
created_at	datetime	创建时间

五、部署与运行

环境要求

- JDK 17+
- Tomcat 10+
- MySQL 8.0+
- IntelliJ IDEA（推荐）

启动步骤

1. 导入数据库：

```
mysql -u root -p < backend/sql/init.sql
mysql -u root -p < backend/sql/test_data.sql
```

2. 配置数据库连接：修改 `DbUtil.java` 或 `web.xml` 中的连接参数

3. 在 IntelliJ IDEA 中配置 Tomcat 10+ 运行配置，部署 `backend_war_exploded` artifact

4. 启动 Tomcat，后端默认运行在 `http://localhost:8080`

日志调试

所有 Servlet 方法均有 `System.out.println` 日志输出（前缀 `[ServletName]`），在 IDEA 控制台查看请求路径、参数、SQL 结果等。

常见问题

1. `jakarta.servlet` **无法解析**：确保 Tomcat 版本 ≥ 10（Jakarta 命名空间）
2. **MySQL 连接失败**：检查 `lost_property` 数据库是否创建，用户密码是否正确
3. **图片上传后 404**：确认 `web/uploads/` 目录存在且有写入权限
4. **JWT 解析失败**：检查 `JwtUtil.java` 中的密钥配置